

SQL-TUTOR: An AI-Powered Interactive SQL Learning Platform

Executive Summary

SQL-TUTOR is an interactive, AI-driven educational platform designed to bridge the gap between theoretical SQL knowledge and practical application. The system addresses the limitations of traditional learning methods—such as static worksheets and a lack of immediate feedback—by providing a real-time, guided environment for practicing SQL queries. By integrating Large Language Models (LLMs) with a robust execution engine, SQL-TUTOR offers personalized hints and feedback, particularly targeting complex concepts like joins, window functions, and Common Table Expressions (CTEs). The platform prioritizes an "eval loop" pedagogy, where students are guided toward the correct answer through incremental hints rather than direct solutions, fostering deeper conceptual understanding.

Problem Statement and Objectives

Traditional SQL education often relies on static documentation and non-interactive worksheets. This approach presents several challenges for students, particularly those at the junior level:

- **Lack of Immediate Feedback:** Students often struggle to identify why a query is failing or producing incorrect results without real-time intervention.
- **Difficulty with Complex Syntax:** Mastering advanced SQL features, such as Common Table Expressions (CTEs), window functions, and complex joins, is difficult without expert guidance.
- **Diminished Engagement:** Without a dynamic feedback loop, students frequently lose interest and fail to establish strong foundational skills. SQL-TUTOR aims to resolve these issues by acting as a "virtual tutor," providing a step-by-step guidance system that tracks progress and rewards achievement.

Core Features and Functionality

The platform is built around a multi-tiered challenge system categorized into **Beginner**, **Intermediate**, and **Advanced** levels.

Key features include:

Challenge System: A curated library of SQL problems stored in JSON format, categorized by difficulty.

Real-time Execution: An integrated environment where users can run SQL queries directly against a sample SQLite database.

Evaluation Logic: An automated system that compares user output with expected output to determine accuracy.

AI Hint System: A progressive assistance tool that provides clues without revealing the final answer.

AI Feedback Engine: A diagnostic tool that explains mistakes in plain language to assist with troubleshooting.

Variant Generator: An AI-powered tool that creates new questions based on the concepts a user is currently practicing.

Gamification | A progress-tracking system that monitors streaks, awards badges, and records solved challenges. |

Technical Architecture

SQL-TUTOR utilizes a modular architecture to ensure a seamless interaction between the user interface and the AI backend.

System Layers

1. **Frontend Layer:** Developed using **Streamlit**, this layer manages the UI, user inputs, and data visualization.
2. **Backend Layer:** A Python-based engine that processes queries and interacts with the **SQLite** database.
3. **Evaluation Layer:** Responsible for comparing student results against the correct query results.
4. **AI Layer:** Powered by **OpenAI or Gemini APIs**, this layer generates hints, provides qualitative feedback, and produces new challenge variants.
5. **Session Layer:** Utilizes Streamlit session states to maintain progress, streaks, and challenge history within a single session.

Tools and Technologies

- **Language:** Python 3.10+
- **Web Framework:** Streamlit
- **Database:** SQLite
- **Data Handling:** Pandas and JSON
- **AI Integration:** OpenAI API / Gemini API

Essential Python Libraries

- **sqlite3:** Core database engine for query execution.
- **pandas:** For rendering query results in a tabular format.
- **re:** Used for SQL validation and pattern matching.
- **json:** To load and manage the challenge library.
- **time & datetime:** To track solving speed and daily usage streaks.
- **pathlib:** For managing project file structures.

AI Implementation and Educational Pedagogy

The defining characteristic of SQL-TUTOR is its "Tutoring Agent" approach. Rather than acting as a simple code generator, the AI serves as a pedagogical guide through three primary capabilities:

1. **Adaptive Hinting:** The system monitors the user's attempt level. If the user fails, it provides hints that become progressively more detailed, ensuring the student still performs the cognitive work required to solve the problem.
2. **Qualitative Feedback:** Beyond stating whether a query is "Right" or "Wrong," the AI explains the logic behind errors, helping students understand the "why" of SQL syntax.
3. **Concept Deep-Diving:** Through the Variant Generator, the AI ensures that if a student struggles with a specific concept (e.g., an INNER JOIN), they can practice multiple variations of that concept to solidify their understanding.

Operational Workflow

The application follows a structured user journey designed to simulate a real-world development environment:

- **Challenge Selection:** The user selects a difficulty level from the sidebar.
- **Schema Review:** Users can expand a section to view the database schema and sample data to understand the table structures.
- **Query Composition:** The user writes their query in a dedicated SQL editor.
- **Execution & Verification:**
 - The user runs the query to see the actual output.
 - The user clicks "Check Answer" to trigger the evaluation logic.
- **Feedback Loop:**
 - **On Success:** The system displays a success message, updates the user's streak, and potentially awards a badge.
 - **On Failure:** The AI intervention begins, offering a hint and an explanation of the error.
- **Progress Reinforcement:** Users can opt to generate a similar challenge to test their mastery of the concept they just solved.

Deployment and Setup

The project is designed to run in a controlled Python environment. The standard deployment workflow includes:

1. **Navigation:** Accessing the project directory (cd sqltutor).
2. **Environment Isolation:** Creating a virtual environment (python -m venv venv).
3. **Activation:** Activating the environment to manage dependencies.
4. **Launch:** Initializing the web application via Streamlit (streamlit run [app.py](#)).

The screenshot shows the SQL Tutor Agent web application running in a browser at localhost:8501. The interface is divided into a sidebar on the left and a main content area on the right.

Sidebar:

- Challenges:** 0/20 solved. Streak: 0d. Hints used: 0.
- Beginner:** A list of challenges: 1. All Active Customers, 2. Count Customers by City, 3. Unique Cities, 4. Total Revenue.

Main Content Area:

- SQL Tutor Agent:** Practice SQL with instant, answer-safe feedback.
- All Active Customers:** A challenge titled "All Active Customers" with a "Beginner" difficulty level. The challenge text is: "Return the first_name and last_name of every customer whose status is 'active'. Order by last_name ascending."
- Schema & sample data:** A section showing the database schema and sample data. The schema is: `CREATE TABLE customers (id INTEGER PRIMARY KEY, first_name TEXT, last_name TEXT, status TEXT, city TEXT);`. The sample data is a table with 5 columns (id, first_name, last_name, status, city) and 5 rows of data.

id	first_name	last_name	status	city
1	Alice	Smith	active	Denver
2	Bob	Jones	inactive	Austin
3	Carol	Brown	active	Denver
4	Dan	Adams	active	Seattle

🔥 Your SQL query

SELECT ...

▶ Run Query

✓ Check Answer

💡 Get Hint

🔗 Similar Challenge

Challenges

0/20 solved

🔥 Streak

0d

💡 Hints used

0

Beginner

1. All Active Customers

2. Count Customers by City

3. Unique Cities

4. Total Revenue

5. High-Value Orders